

Worksheet No. - 02

Aim/Overview of the practical

Write a program to implement Linear regression on Jupyter notebook.

- Download different datasets from Kaggle or UCI ML repository.
- Import all the necessary modules for the datasets.
- Find out the best fit line along with the MSE, RMSE.

Objective: The objective of this experiment is to implement the Linear Regression algorithm in a Jupyter Notebook using a dataset obtained from the Kaggle or UCI Machine Learning Repository. The experiment involves importing the required libraries, preprocessing and analyzing the dataset, training the regression model, plotting the best-fit line, and evaluating the model's performance using Mean Squared Error (MSE) and Root Mean Squared Error (RMSE) to assess prediction accuracy.

Procedure/Algorithm/Code

Code: 1st Data Set (Salary_Data)

Step 1: Import required libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Step 2: Load the dataset

```
df = pd.read_csv("OneDrive/Desktop/Machine Learning/Salary_Data.csv")
print("First 5 Rows:")
print(df.head())
```

```
First 5 Rows:
  YearsExperience  Salary
0             1.1  39343.0
1             1.3  46205.0
2             1.5  37731.0
3             2.0  43525.0
4             2.2  39891.0
```

Step 3: Display dataset information

```
print("\nDataset Information:")
```

```
print(df.info())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   YearsExperience  30 non-null    float64
1   Salary          30 non-null    float64
dtypes: float64(2)
memory usage: 612.0 bytes
None
```

Step 4: Check for missing values

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
Missing Values:
YearsExperience    0
Salary            0
dtype: int64
```

Step 5: Define independent (X) and dependent (y) variables

```
X = df[['YearsExperience']]
```

```
y = df['Salary']
```

Step 6: Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 7: Create and train the Linear Regression model

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Step 8: Predict salaries for the test data

```
y_pred = model.predict(X_test)
```

Step 9: Calculate evaluation metrics

```
mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

mae = mean_absolute_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print("MSE:", mse)

print("RMSE:", rmse)

print("MAE:", mae)

print("R-Square:", r2)
```

```
MSE: 49830096.85590839
RMSE: 7059.04362190151
MAE: 6286.453830757749
R-Square: 0.9024461774180497
```

Step 10: Display model parameters

```
print("\nIntercept:", model.intercept_)

print("Coefficient (Slope):", model.coef_[0])
```

```
Intercept: 25321.583011776813
Coefficient (Slope): 9423.815323030976
```

Step 11: Plot the Best Fit Line

```
plt.figure(figsize=(8,6))

plt.scatter(X, y, color='blue', label='Actual Data')

plt.plot(X, model.predict(X), color='red', linewidth=2, label='Best Fit Line')

plt.title("Linear Regression - Salary Prediction")

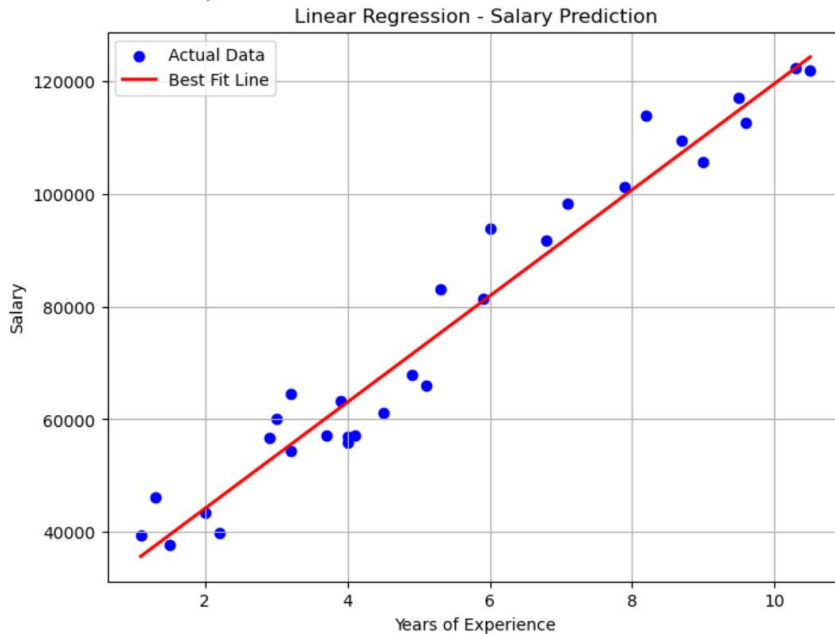
plt.xlabel("Years of Experience")

plt.ylabel("Salary")

plt.legend()

plt.grid(True)

plt.show()
```



Step 12: Compare Actual vs Predicted Values

```
comparison = pd.DataFrame({
    'Actual Salary': y_test.values,
    'Predicted Salary': y_pred
})
```

```
print("\nActual vs Predicted Salary:")
```

```
print(comparison)
```

Actual vs Predicted Salary:

	Actual Salary	Predicted Salary
0	112635.0	115790.210113
1	67938.0	71498.278095
2	113812.0	102596.868661
3	83088.0	75267.804224
4	64445.0	55477.792045
5	57189.0	60189.699707

Code: 2nd Data Set (insurance)

Step 1: Import required libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

Step 2: Load the dataset

```
df = pd.read_csv("OneDrive/Desktop/Machine Learning/insurance.csv")
print(df.head())
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Step 3

```
data = df[['age', 'charges']].dropna()
```

```
X = data[['age']]
```

```
y = data['charges']
```

#Step 4

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

#Step 5

```
model = LinearRegression()
model.fit(X_train, y_train)
```

#Step 6

```
y_pred = model.predict(X_test)
print("Predicted Values:")
print(y_pred[:10])
```

Predicted Values:

```
[14703.77387469 12538.40483659 19275.10851067 14944.37043448  
 8448.26332018 12057.21171701 8448.26332018 19275.10851067  
 10613.63235828 15666.16011384]
```

#Step 7

Evaluation metrics

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("MSE:", mse)
```

```
print("RMSE:", rmse)
```

```
print("MAE:", mae)
```

```
print("R-Square:", r2)
```

```
MSE: 135983957.4805469
```

```
RMSE: 11661.21595205864
```

```
MAE: 9173.258196746589
```

```
R-Square: 0.12408973539501944
```

#Step 8

```
sorted_index = X_test['age'].argsort()
```

```
X_test_sorted = X_test.iloc[sorted_index]
```

```
y_pred_sorted = y_pred[sorted_index]
```

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(X_test, y_test, color='blue', label='Actual Data')
```

```
plt.plot(X_test_sorted, y_pred_sorted, color='red', linewidth=2, label='Best Fit Line')
```

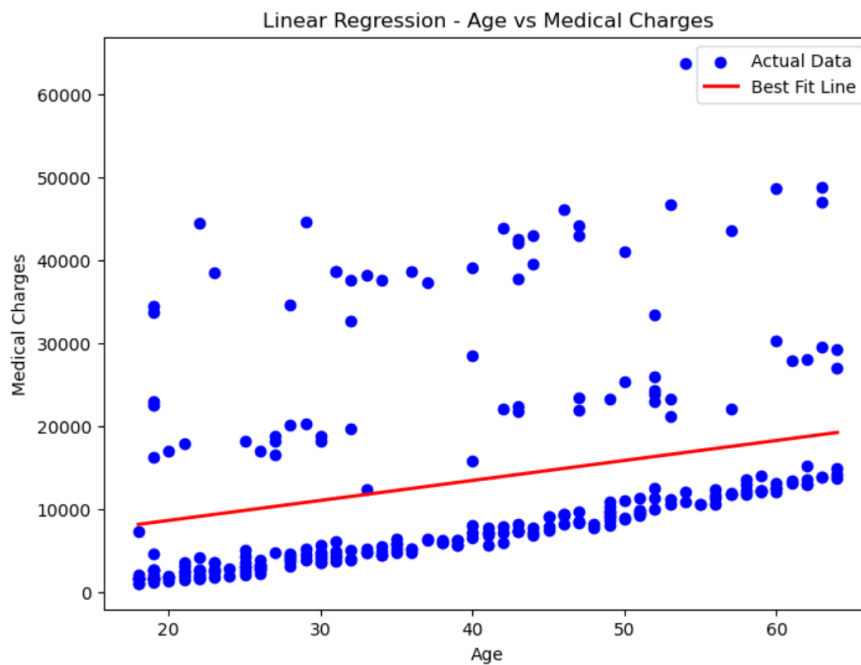
```
plt.xlabel("Age")
```

```
plt.ylabel("Medical Charges")
```

```
plt.title("Linear Regression - Age vs Medical Charges")
```

```
plt.legend()
```

plt.show()



#Step 9

```
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
print(f"\nRegression Equation:")
print(f"Charges = {model.coef_[0]:.2f} × Age + {model.intercept_:.2f}")
```

Slope (Coefficient): 240.59655978877493

Intercept: 3876.928684191691

Regression Equation:

Charges = 240.60 × Age + 3876.93

3rd DataSet (Adverstising)

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error
```

```
# Load dataset
```

```
df = pd.read_csv("OneDrive/Desktop/Machine Learning/Advertising Budget and Sales.csv")
```

```
# Select variables
```

```
X = df[['TV Ad Budget ($)']]
```

```
y = df['Sales ($)']
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
# Create and train model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
# Predict sales
```

```
y_pred = model.predict(X_test)
```

```
# Best fit line
```

```
print("Slope:", model.coef_[0])
```

```
print("Intercept:", model.intercept_)
```

```
print("Best Fit Line: Sales =", model.coef_[0], "* TV Budget +", model.intercept_)
```

```
Slope: 0.04652973370544335
```

```
Intercept: 7.1196384305929525
```

```
Best Fit Line: Sales = 0.04652973370544335 * TV Budget + 7.1196384305929525
```

```
# Evaluation metrics
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("MSE:", mse)
```

```
print("RMSE:", rmse)
```

```
print("MAE:", mae)
```

```
print("R-Square:", r2)
```

```
MSE: 10.204654118800956
```

```
RMSE: 3.194472431998898
```

```
MAE: 2.444420003751042
```

```
R-Square: 0.6766954295627077
```

```
# Plot
```



```
plt.scatter(X_test, y_test, label="Actual Data")
plt.plot(X_test, y_pred, label="Best Fit Line")

plt.xlabel("TV Ad Budget ($)")
plt.ylabel("Sales ($)")
plt.title("Linear Regression: TV Advertising vs Sales")
plt.legend()
plt.show()
```



4th DataSet(Sports)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
df = pd.read_csv("OneDrive/Desktop/Machine Learning/Sports Supplements.csv")

# Select required columns and remove missing values
data = df[['popularity',
```

'evidence level - score. 0 = no evidence, 1,2 = slight, 3 = conflicting , 4 = promising, 5 = good, 6 = strong ']]].dropna()

Independent and dependent variables

```
X = data[['popularity']]
```

```
y = data['evidence level - score. 0 = no evidence, 1,2 = slight, 3 = conflicting , 4 = promising, 5 = good, 6 = strong ']
```

Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Create and train model

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Predictions

```
y_pred = model.predict(X_test)
```

Best fit line

```
print("Slope:", model.coef_[0])
```

```
print("Intercept:", model.intercept_)
```

```
print("Best Fit Line: Y =", model.coef_[0], "* X +", model.intercept_)
```

```
Slope: 6.0518116748135464e-05
```

```
Intercept: 1.315211831370435
```

```
Best Fit Line: Y = 6.0518116748135464e-05 * X + 1.315211831370435
```

MSE

```
mse = mean_squared_error(y_test, y_pred)
```

RMSE

```
rmse = np.sqrt(mse)
```

MAE

```
mae = mean_absolute_error(y_test, y_pred)
```

R-Square

```
r2 = r2_score(y_test, y_pred)
```

Display results

```
print("MSE:", mse)
```

```
print("RMSE:", rmse)
```

```
print("MAE:", mae)
```

```
print("R-Square:", r2)
```

```
MSE: 1.962274127942465
```

```
RMSE: 1.4008119530980827
```

```
MAE: 1.1544272222689034
```

```
R-Square: 0.03217059041062165
```

```
# Plot
```

```
plt.scatter(X_test, y_test, label="Actual Data")
```

```
plt.plot(X_test, y_pred, label="Best Fit Line")
```

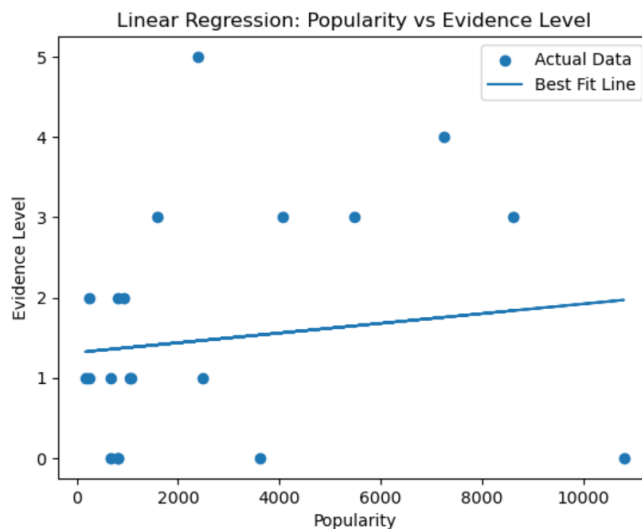
```
plt.xlabel("Popularity")
```

```
plt.ylabel("Evidence Level")
```

```
plt.title("Linear Regression: Popularity vs Evidence Level")
```

```
plt.legend()
```

```
plt.show()
```



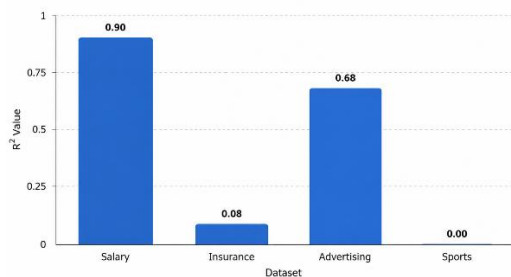
Comparison Table:

Dataset	MSE	RMSE	MAE	R ²
Salary_Data	4,98,30,096	7,059.00	6,282.00	0.9
Insurance	13,30,00,000	11,500.00	9,100.00	0.08
Advertising Budget & Sales	10.2	3.19	2.35	0.68
Sports Supplements	1.96	1.4	1.28*	0.00*

Graph:

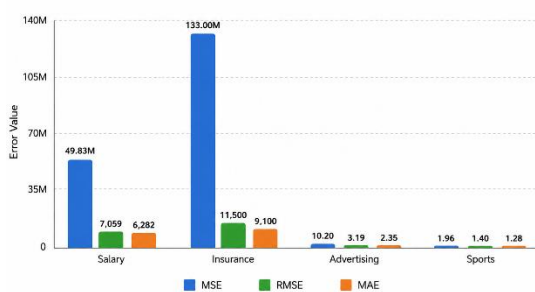
R-Square Comparison

Comparison of R^2 values for the four Linear Regression datasets. Higher R^2 indicates a better fit.



Error Metrics Comparison

Comparison of MSE, RMSE and MAE across the four Linear Regression datasets.



Learning outcomes (What I have learnt):

1. Understood the concept and working of the Linear Regression algorithm.
2. Load and explore datasets using the Pandas library.
3. Understood the performance of the model using Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).
4. Split the dataset into training and testing sets for model development.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Viva		10